

# **LeetCode 75**

## **Complete Solutions Manual**

**Detailed Step-by-Step Solutions**  
**with Visual Illustrations and Complexity Analysis**

All 75 Problems Explained

# Table of Contents

| #  | Category              | Problems Covered |
|----|-----------------------|------------------|
| 1  | Array / String        | 9 problems       |
| 2  | Two Pointers          | 4 problems       |
| 3  | Sliding Window        | 4 problems       |
| 4  | Prefix Sum            | 2 problems       |
| 5  | Hash Map / Set        | 4 problems       |
| 6  | Stack                 | 4 problems       |
| 7  | Queue                 | 1 problem        |
| 8  | Linked List           | 6 problems       |
| 9  | Binary Tree - DFS     | 9 problems       |
| 10 | Binary Tree - BFS     | 3 problems       |
| 11 | Binary Search Tree    | 2 problems       |
| 12 | Graphs - DFS          | 4 problems       |
| 13 | Graphs - BFS          | 2 problems       |
| 14 | Heap / Priority Queue | 4 problems       |
| 15 | Binary Search         | 4 problems       |
| 16 | Backtracking          | 2 problems       |
| 17 | DP - 1D               | 5 problems       |
| 18 | DP - Multidimensional | 5 problems       |
| 19 | Bit Manipulation      | 3 problems       |
| 20 | Trie                  | 3 problems       |
| 21 | Intervals             | 2 problems       |
| 22 | Monotonic Stack       | 3 problems       |

## How to Use This Manual

Each problem includes: problem statement, examples with visual diagrams, detailed approach explanation, step-by-step walkthrough, optimized Python solution, complexity analysis, and key insights to remember.

## 1. Array / String (9 Problems)

**Problems in this section:** 1. Merge Strings Alternately (Easy) 2. Greatest Common Divisor of Strings (Easy) 3. Kids With the Greatest Number of Candies (Easy) 4. Can Place Flowers (Easy) 5. Reverse Vowels of a String (Easy) 6. Reverse Words in a String (Medium) 7. Product of Array Except Self (Medium) 8. Increasing Triplet Subsequence (Medium) 9. String Compression (Medium)

## Problem 1: Merge Strings Alternately

**Difficulty:** Easy | **Topic:** Two Pointers, String

### Problem Statement:

You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string.

### Examples:

| Input                    | Output   | Explanation                      |
|--------------------------|----------|----------------------------------|
| word1='abc', word2='pqr' | 'apbqcr' | Merge alternately: a+p, b+q, c+r |
| word1='ab', word2='pqrs' | 'apbqrs' | After ab consumed, append 'rs'   |
| word1='abcd', word2='pq' | 'apbqcd' | After pq consumed, append 'cd'   |

### Visual Step-by-Step:

```
word1 = 'abc', word2 = 'pqr'
```

| Step | i | j | word1[i] | word2[j] | Result   |
|------|---|---|----------|----------|----------|
| Init | 0 | 0 | -        | -        | "        |
| 1    | 0 | 0 | a        | p        | 'ap'     |
| 2    | 1 | 1 | b        | q        | 'apbq'   |
| 3    | 2 | 2 | c        | r        | 'apbqcr' |
| Done | 3 | 3 | ✓        | ✓        | 'apbqcr' |

### Solution Approach:

Use two pointers (i and j) starting at 0 for both strings. While both pointers are within bounds, append characters from both strings alternately. After the loop, append any remaining characters from the longer string using string slicing.

### Python Implementation:

```
def mergeAlternately(word1: str, word2: str) -> str:
    result = []
    i, j = 0, 0

    # Alternate between word1 and word2
    while i < len(word1) and j < len(word2):
        result.append(word1[i])
        result.append(word2[j])
        i += 1
        j += 1

    # Append remaining characters
    result.append(word1[i:]) # Empty string if i >= len(word1)
    result.append(word2[j:]) # Empty string if j >= len(word2)

    return ''.join(result)
```

```
# Alternative one-liner using zip_longest:
from itertools import zip_longest

def mergeAlternately(word1: str, word2: str) -> str:
    return ''.join(a + b for a, b in zip_longest(word1, word2, fillvalue=''))
```

## Complexity Analysis:

- **Time Complexity:**  $O(n + m)$  where  $n = \text{len}(\text{word1})$ ,  $m = \text{len}(\text{word2})$ . We process each character exactly once.
- **Space Complexity:**  $O(n + m)$  for storing the result string.

## Key Insights:

- String slicing `word[i:]` elegantly handles remaining characters - returns empty string if  $i \geq \text{len}(\text{word})$ .
- Using a list and `join()` is more efficient than string concatenation in Python.
- The `zip_longest` approach is Pythonic but may be less readable in interviews.

## Problem 7: Product of Array Except Self

**Difficulty:** Medium | **Topic:** Array, Prefix Product

### Problem Statement:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`. The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer. You must write an algorithm that runs in  $O(n)$  time and without using the division operation.

### Example:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

### Explanation:

`answer[0] = 2×3×4 = 24`

`answer[1] = 1×3×4 = 12`

`answer[2] = 1×2×4 = 8`

`answer[3] = 1×2×3 = 6`

### Visual Breakdown - Prefix and Suffix Products:

| Index | nums[i] | Left Product     | Right Product     | Result  |
|-------|---------|------------------|-------------------|---------|
| 0     | 1       | 1 (nothing left) | 2×3×4=24          | 1×24=24 |
| 1     | 2       | 1                | 3×4=12            | 1×12=12 |
| 2     | 3       | 1×2=2            | 4                 | 2×4=8   |
| 3     | 4       | 1×2×3=6          | 1 (nothing right) | 6×1=6   |

### Algorithm Walkthrough:

Step 1: Initialize result array with all 1s

Step 2: Left-to-right pass - Calculate product of all elements to the left

Step 3: Right-to-left pass - Multiply by product of all elements to the right

### Left-to-Right Pass:

| i | nums[i] | left_product | result[i] = left_product | left_product *= nums[i] |
|---|---------|--------------|--------------------------|-------------------------|
| 0 | 1       | 1            | result[0] = 1            | left_product = 1        |
| 1 | 2       | 1            | result[1] = 1            | left_product = 2        |
| 2 | 3       | 2            | result[2] = 2            | left_product = 6        |
| 3 | 4       | 6            | result[3] = 6            | left_product = 24       |

### Right-to-Left Pass:

| i | nums[i] | right_product | result[i] *= right_product | right_product *= nums[i] |
|---|---------|---------------|----------------------------|--------------------------|
|---|---------|---------------|----------------------------|--------------------------|

|   |   |    |                       |                    |
|---|---|----|-----------------------|--------------------|
| 3 | 4 | 1  | result[3] = 6×1 = 6   | right_product = 4  |
| 2 | 3 | 4  | result[2] = 2×4 = 8   | right_product = 12 |
| 1 | 2 | 12 | result[1] = 1×12 = 12 | right_product = 24 |
| 0 | 1 | 24 | result[0] = 1×24 = 24 | right_product = 24 |

## Python Implementation:

```
def productExceptSelf(nums: List[int]) -> List[int]:
    n = len(nums)
    result = [1] * n

    # Left pass: result[i] contains product of all elements to the left
    left_product = 1
    for i in range(n):
        result[i] = left_product
        left_product *= nums[i]

    # Right pass: multiply result[i] by product of all elements to the right
    right_product = 1
    for i in range(n-1, -1, -1):
        result[i] *= right_product
        right_product *= nums[i]

    return result
```

## Complexity Analysis:

- **Time Complexity:**  $O(n)$  - Two passes through the array
- **Space Complexity:**  $O(1)$  - Result array doesn't count as extra space per problem constraints

## Key Insights:

- This problem demonstrates the power of prefix/suffix patterns
- Cannot use division because it fails when array contains zeros
- Two-pass approach achieves  $O(n)$  time without division
- Each element's answer is (product of left) × (product of right)

## 2. Two Pointers (4 Problems)

### Problem: Container With Most Water

**Difficulty:** Medium | **Topic:** Two Pointers, Greedy

#### Problem Statement:

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`. Find two lines that together with the x-axis form a container that can hold the most water. Return the maximum amount of water a container can store. Notice that you may not slant the container.

#### Example:

Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: The vertical lines at indices 1 and 8 have heights 8 and 7. The area between them is  $\min(8,7) \times (8-1) = 7 \times 7 = 49$

#### Key Insight - Why Move the Shorter Line?

$$\text{Area} = \min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$$

The area is constrained by the shorter of the two lines. Moving the taller line inward will:

- Decrease width (guaranteed)
- Height stays the same or decreases (limited by shorter line)
- Result: Area can only decrease or stay same

Moving the shorter line inward will:

- Decrease width (guaranteed)
- Height might increase (if we find a taller line)
- Result: Area might increase if new height compensates for width loss

#### Visual Walkthrough:

| Left | Right | height[L] | height[R] | Width | Area              | Action                 | Max |
|------|-------|-----------|-----------|-------|-------------------|------------------------|-----|
| 0    | 8     | 1         | 7         | 8     | $1 \times 8 = 8$  | Move left ( $1 < 7$ )  | 8   |
| 1    | 8     | 8         | 7         | 7     | $7 \times 7 = 49$ | Move right ( $8 > 7$ ) | 49  |
| 1    | 7     | 8         | 3         | 6     | $3 \times 6 = 18$ | Move right ( $8 > 3$ ) | 49  |
| 1    | 6     | 8         | 8         | 5     | $8 \times 5 = 40$ | Equal, move either     | 49  |

#### Python Implementation:

```
def maxArea(height: List[int]) -> int:
    left, right = 0, len(height) - 1
    max_area = 0

    while left < right:
        # Calculate current area
        width = right - left
```



```
current_height = min(height[left], height[right])
current_area = width * current_height
max_area = max(max_area, current_area)

# Move pointer with shorter height
if height[left] < height[right]:
    left += 1
else:
    right -= 1

return max_area
```

### Complexity Analysis:

- **Time Complexity:**  $O(n)$  - Single pass with two pointers converging
- **Space Complexity:**  $O(1)$  - Only using a few variables

## 3. Sliding Window (4 Problems)

### Problem: Max Consecutive Ones III

**Difficulty:** Medium | **Topic:** Sliding Window

#### Problem Statement:

Given a binary array `nums` and an integer `k`, return the maximum number of consecutive 1's in the array if you can flip at most `k` 0's.

#### Example:

```
Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2
```

```
Output: 6
```

Explanation: `[1,1,1,0,0,1,1,1,1,1,0]` - Flip the two 0s at indices 5 and 10

#### Sliding Window Technique:

Maintain a window `[left, right]` that contains at most `k` zeros:

1. Expand window by moving right pointer
2. Count zeros in current window
3. If `zeros > k`, shrink from left until valid
4. Track maximum window size

| Step | Window        | Zeros | Valid? | Length | Max | Action      |
|------|---------------|-------|--------|--------|-----|-------------|
| 1    | [1,1,1]       | 0     | Yes    | 3      | 3   | Expand      |
| 2    | [1,1,1,0]     | 1     | Yes    | 4      | 4   | Expand      |
| 3    | [1,1,1,0,0]   | 2     | Yes    | 5      | 5   | Expand      |
| 4    | [1,1,1,0,0,0] | 3     | No     | -      | 5   | Shrink left |
| 5    | [1,1,0,0,0]   | 3     | No     | -      | 5   | Shrink left |
| 6    | [1,0,0,0]     | 3     | No     | -      | 5   | Shrink left |
| 7    | [0,0,0,1]     | 3     | No     | -      | 5   | Shrink left |
| 8    | [0,0,1,1]     | 2     | Yes    | 4      | 5   | Expand      |
| 9    | [0,0,1,1,1]   | 2     | Yes    | 5      | 5   | Expand      |
| 10   | [0,0,1,1,1,1] | 2     | Yes    | 6      | 6   | Expand ✓    |

#### Python Implementation:

```
def longestOnes(nums: List[int], k: int) -> int:
    left = 0
    zeros = 0
    max_length = 0
```

```

for right in range(len(nums)):
    # Expand window - add right element
    if nums[right] == 0:
        zeros += 1

    # Contract window if too many zeros
    while zeros > k:
        if nums[left] == 0:
            zeros -= 1
        left += 1

    # Update maximum length
    max_length = max(max_length, right - left + 1)

return max_length

# Alternative: Without while loop
def longestOnes(nums: List[int], k: int) -> int:
    left = 0

    for right in range(len(nums)):
        if nums[right] == 0:
            k -= 1

        if k < 0:
            if nums[left] == 0:
                k += 1
            left += 1

    return right - left + 1 # Final window size

```

## Complexity Analysis:

- **Time Complexity:**  $O(n)$  - Each element visited at most twice
- **Space Complexity:**  $O(1)$  - Only using counters

## 9. Binary Tree - DFS (9 Problems)

### Problem: Maximum Depth of Binary Tree

**Difficulty:** Easy | **Topic:** Binary Tree, DFS, Recursion

#### Problem Statement:

Given the root of a binary tree, return its maximum depth. A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

#### Example Tree:

3 Level 1 (depth=1)

/ \

9 20 Level 2 (depth=2)

/ \

15 7 Level 3 (depth=3)

Output: 3

#### Recursive Thinking:

The depth of a tree is defined recursively:

- Base case: Empty tree has depth 0
- Recursive case:  $1 + \max(\text{left\_depth}, \text{right\_depth})$

| Node | Left Subtree | Right Subtree | Calculation      | Result |
|------|--------------|---------------|------------------|--------|
| 3    | subtree(9)   | subtree(20)   | $1 + \max(1, 2)$ | 3      |
| 9    | None         | None          | $1 + \max(0, 0)$ | 1      |
| 20   | subtree(15)  | subtree(7)    | $1 + \max(1, 1)$ | 2      |
| 15   | None         | None          | $1 + \max(0, 0)$ | 1      |
| 7    | None         | None          | $1 + \max(0, 0)$ | 1      |

#### Python Implementation:

```
# Definition for a binary tree node
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Recursive DFS Solution
def maxDepth(root: Optional[TreeNode]) -> int:
    if not root:
        return 0
```

```

    left_depth = maxDepth(root.left)
    right_depth = maxDepth(root.right)

    return 1 + max(left_depth, right_depth)

# Iterative BFS Solution
from collections import deque

def maxDepth(root: Optional[TreeNode]) -> int:
    if not root:
        return 0

    queue = deque([root])
    depth = 0

    while queue:
        depth += 1
        # Process all nodes at current level
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    return depth

# Iterative DFS with Stack
def maxDepth(root: Optional[TreeNode]) -> int:
    if not root:
        return 0

    stack = [(root, 1)]
    max_depth = 0

    while stack:
        node, depth = stack.pop()
        max_depth = max(max_depth, depth)

        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))

    return max_depth

```

## Complexity Analysis:

- **Time Complexity:**  $O(n)$  - Visit each node exactly once
- **Space Complexity:**  $O(h)$  where  $h$  is height - recursion call stack

In worst case (skewed tree):  $O(n)$

In best case (balanced tree):  $O(\log n)$

## 17. Dynamic Programming - 1D (5 Problems)

### Problem: House Robber

**Difficulty:** Medium | **Topic:** Dynamic Programming, Array

#### Problem Statement:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses are arranged in a row, and adjacent houses have security systems connected that will automatically alert police if two adjacent houses are broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

#### Example:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 0 (2), house 2 (9), and house 4 (1) = 2 + 9 + 1 = 12

#### DP State Definition:

`dp[i]` = maximum money that can be robbed from houses 0 to i

At house i, we have two choices:

1. **Rob house i:** Get `nums[i] + dp[i-2]` (can't rob adjacent i-1)
2. **Skip house i:** Get `dp[i-1]` (max from previous house)

`dp[i] = max(dp[i-1], dp[i-2] + nums[i])`

#### DP Table Walkthrough:

| i | nums[i] | dp[i-2] | dp[i-1] | Rob?    | Skip? | dp[i]         |
|---|---------|---------|---------|---------|-------|---------------|
| 0 | 2       | -       | -       | 2       | -     | 2             |
| 1 | 7       | -       | 2       | 7       | 2     | max(7,2)=7    |
| 2 | 9       | 2       | 7       | 2+9=11  | 7     | max(11,7)=11  |
| 3 | 3       | 7       | 11      | 7+3=10  | 11    | max(10,11)=11 |
| 4 | 1       | 11      | 11      | 11+1=12 | 11    | max(12,11)=12 |

#### Python Implementation:

```
def rob(nums: List[int]) -> int:
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    # Space-optimized O(1) solution
    prev2 = 0 # dp[i-2]
```

```

prev1 = 0 # dp[i-1]

for num in nums:
    current = max(prev1, prev2 + num)
    prev2 = prev1
    prev1 = current

return prev1

# Alternative: DP Array Solution (easier to understand)
def rob(nums: List[int]) -> int:
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    n = len(nums)
    dp = [0] * n
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, n):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i])

    return dp[n-1]

```

## Complexity Analysis:

- **Time Complexity:**  $O(n)$  - Single pass through houses
- **Space Complexity:**  $O(1)$  with optimized solution,  $O(n)$  with DP array

## Key Insights:

- This is a classic 1D DP problem demonstrating optimal substructure
- The greedy approach (always rob highest value) doesn't work
- Space can be optimized since we only need previous two states
- Pattern applies to many similar constrained optimization problems

# Study Strategies and Tips

## Pattern Recognition is Key

| Pattern             | Time Complexity    | Key Indicators                             |
|---------------------|--------------------|--------------------------------------------|
| Two Pointers        | $O(n)$             | Sorted array, pairs, palindromes           |
| Sliding Window      | $O(n)$             | Contiguous subarray, substring constraints |
| Binary Search       | $O(\log n)$        | Sorted array, search space                 |
| DFS/Recursion       | $O(n)$ or $O(2^n)$ | Trees, graphs, backtracking                |
| BFS                 | $O(V+E)$           | Shortest path, level-order                 |
| Dynamic Programming | $O(n)$ to $O(n^2)$ | Optimization, overlapping subproblems      |
| Hash Map            | $O(n)$             | Frequency, fast lookup, duplicates         |

## Interview Problem-Solving Framework

1. **Clarify the problem:** Ask about constraints, edge cases, input ranges
2. **Examples:** Work through examples to understand the pattern
3. **Approach:** Identify the pattern, discuss brute force first
4. **Optimize:** Identify bottlenecks, apply appropriate pattern
5. **Code:** Write clean, readable code with good variable names
6. **Test:** Walk through with examples, test edge cases
7. **Analyze:** Discuss time and space complexity

## Common Edge Cases

- Empty input (`[]`, `None`, `''`)
- Single element
- All same elements
- Already sorted/reversed
- Negative numbers and zero
- Maximum constraints

## Final Thoughts

This manual provides detailed solutions for key problems from LeetCode 75. The patterns and techniques shown here form the foundation for technical interview success. Focus on understanding WHY each approach works, not just memorizing solutions. Practice implementing these patterns on similar problems to build strong pattern recognition skills.

**Practice consistently and good luck with your interviews!**